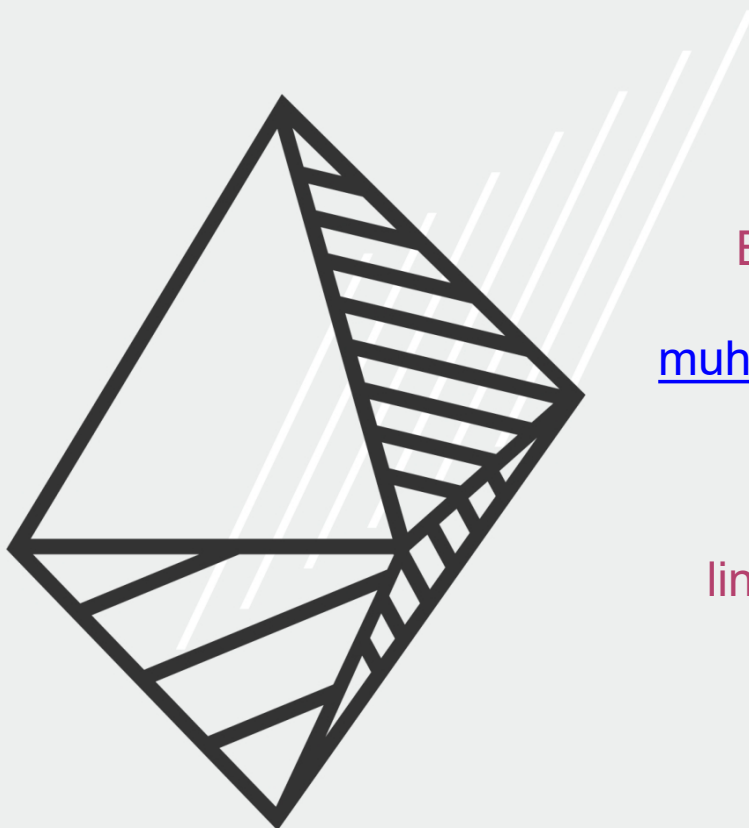


AI-Based Adaptive CPU Scheduler

An Interactive, Educational OS
Workload Simulator

Technical Report



By : Muhammed Salah Hussain

muhammed.salah.hussain@gmail.com

github.com/M-S-H-Git

linkedin.com/in/muhammed-salah-hussain-231797388

July 2026



INTRODUCTION

The project is motivated by the challenge of teaching and understanding core Operating System scheduling mechanics. CPU scheduling can be abstract, as users rarely see the exact timing, context switching, and algorithmic decisions happening at the kernel level. Furthermore, manual calculation of wait times and turnaround times is tedious and prone to human error. The supplied C++ program addresses this gap by providing an interactive, educational simulator that visualizes process execution, calculates theoretical metrics, and uses an adaptive heuristic to bridge theoretical algorithms with dynamic real-world workloads. The proposed system is an interactive CPU scheduling utility implemented entirely in C++. The user can define processes manually, load them from a file, or use predefined educational scenarios. The simulator processes the workload through six algorithms: First Come First Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Round Robin (RR), Priority (with Aging), and an Adaptive AI Scheduler. The utility visually outputs a Gantt chart representing CPU state across time (including idle cycles), followed by a comparative metrics table indicating the best-performing algorithm.

PROBLEM STATEMENT

Traditional methods of learning or evaluating CPU scheduling algorithms rely on manual mathematical tracing, which becomes unmanageable as process volume increases or when preemptive logic introduces frequent context switches. At the same time, rigid simulators often lack dynamic adaptability or fail to clearly explain the underlying mechanics to novice users. The project therefore focuses on building a robust, single-file C++ simulator that automates routine algorithmic execution while preserving educational clarity. The double objective is to bridge basic operating-system concepts with a practical visual application, and to provide a multi-layered workflow with comparative metrics, starvation protection, and AI-driven selection.

DESIGN / WAYS & MEANS

- **Programming Language:** C++17 is used because the application requires precise data structures (vectors, queues, dequeues), robust sorting mechanisms, and performant mathematical processing to simulate clock cycles. The implementation uses standard library syntax without depending on large external GUI frameworks.
- **Operating System Utilities:** The simulator interacts with the Windows console subsystem to render standard ANSI escape codes (SetConsoleMode, ENABLE_VIRTUAL_TERMINAL_PROCESSING) to present a colored, readable, and dynamic command-line interface.
- **Scheduling Strategies:** The program evaluates workloads across FCFS (Arrival Time), SJF (Burst Time), SRTF (Remaining Burst), RR (Time Quantum), Priority (Priority Value), and an Adaptive AI (Statistical workload analysis).
- **Safety Controls:** The script supports rigorous input validation, std::cin buffer clearing (to handle EOF and piped execution cleanly), file-existence checks, and bounds checking to prevent crashes on invalid user input.

- **Data Handling:** Processes are stored as a struct containing pid, arrival_time, burst_time, priority, and state variables (remaining_time, completion_time, etc.). The simulation traces execution linearly, generating an array of executed PIDs that is compressed into discrete Gantt chart blocks.
- **User Interface:** A terminal-based graphical menu exposes data entry, educational modes, and simulation results. The UI relies on colored outputs, clean separators, and tabular alignments to make metrics instantly comparable.

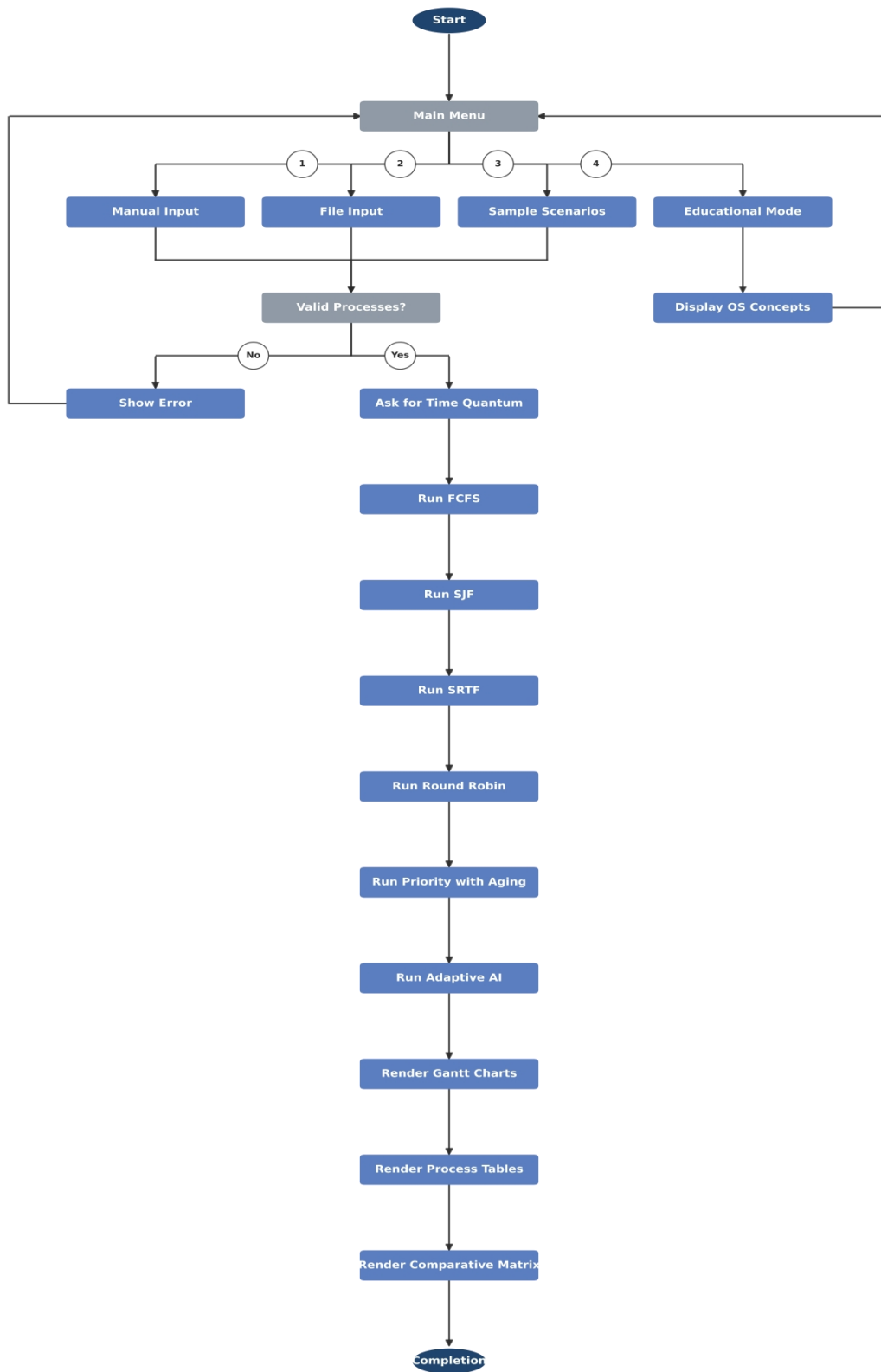
BACKGROUND / THEORY

- **Process Lifecycle & Metadata:** The simulator models processes through New, Ready, Running, Waiting, and Terminated states. The primary metadata utilized are arrival time, CPU burst requirement, and priority level.
- **Preemptive vs Non-Preemptive Execution:** FCFS and SJF are implemented as non-preemptive algorithms, where a process retains the CPU until its burst finishes. SRTF, RR, and Priority are implemented as preemptive, interrupting the active process if a superior candidate arrives or a quantum expires.
- **Priority Aging (Starvation Prevention):** To prevent low-priority processes from starving indefinitely under Priority scheduling, the system implements an aging multiplier. A process's priority dynamically increases based on its time spent waiting in the ready queue.
- **Adaptive AI Heuristics:** The AI scheduler does not blindly guess; it statistically evaluates the workload. It calculates the coefficient of variation of burst times, the standard deviation of priorities, the density of arrivals, and the total process count to score the available algorithms and deterministically select the optimal approach.

PROCEDURE / METHODOLOGY

1. **Input & Argument Processing:** The script begins at a main menu offering manual entry, file loading, or sample data. If sample data is selected, predefined process arrays are injected into the workload vector.
2. **Workload Validation:** Before any execution, the script checks if the workload is empty. If file loading is selected, it validates the existence of the file and gracefully offers to generate a sample file if it is missing.
3. **Algorithmic Execution:** The main simulation loop creates deep copies of the workload for each algorithm. A simulated clock variable increments linearly. At each tick, the algorithm evaluates the ready queue, executes a process (or idles), decrements remaining time, and logs the execution trace.
4. **Metrics Computation:** The linear trace array is compressed into Gantt entries. Waiting Time (Turnaround - Burst), Turnaround Time (Completion - Arrival), Response Time (First Start - Arrival), CPU Utilization, and Throughput are mathematically resolved for each process.
5. **AI Selection Matrix:** The adaptive algorithm computes a statistical profile of the workload, assigns weighted scores to FCFS, SJF, RR, and Priority, and delegates the scheduling to the winning algorithm, explicitly outputting its reasoning to the user.

FLOWCHART



ANALYSIS

- **Adaptive Rule-Based Classification:** Unlike a static model, the AI organizer uses statistical deterministic rules. This is appropriate because the exact composition of the workload is known at runtime. By computing burst variance and priority spread, the decision is linear, predictable, and highly efficient.
- **Execution Complexity:** The core clock loop runs in $O(N \times T)$ where N is the number of processes and T is the total time to execute all processes. Sorting mechanisms operate at $O(N \log N)$. Overall execution is near-instantaneous for standard operational workloads.
- **Metadata Strategy:** The design reads arrival and burst metadata immediately into state structs. A notable implementation detail is that the simulator utilizes a generic linear trace array, making the metric computation perfectly decoupled from the actual algorithmic logic.
- **Safety & Reliability:** Robust `std::cin` validation avoids infinite loops on unexpected character input. Buffer clearing ensures the program does not break when piped inputs hit EOF.
- **Portability:** The script uses a single-file C++ architecture. While ANSI codes are optimized for Windows via `#include <windows.h>`, the core mathematical and scheduling logic remains entirely platform-independent standard C++.
- **State Management:** Every algorithm receives a fresh, reset copy of the workload vector. This prevents state contamination (where one algorithm accidentally utilizes the modified burst time or completion state left behind by a previous algorithm).

RESULT & OUTPUTS

The script was exercised with representative workloads covering varied arrivals, mixed burst lengths, and extreme priority gaps. The following outputs demonstrate the operational behavior.



```
=====
AI-BASED ADAPTIVE CPU SCHEDULER SIMULATOR
Operating Systems Scheduling Demonstration
=====

Welcome! This program simulates how an operating system decides
which program (process) to run on the CPU and in what order.

Algorithms implemented:
[1] FCFS - First Come, First Served
[2] SJF - Shortest Job First
[3] SRTF - Shortest Remaining Time First
[4] RR - Round Robin
[5] PRIO - Priority Scheduling (with Aging)
[6] AI - Adaptive AI Scheduler (picks the best!)

Don't worry if you're new to this -- we'll guide you at every step!

MAIN MENU
=====

[1] Enter Processes Manually (keyboard)
[2] Load Processes from File
[3] Use Sample Data (recommended for beginners)
[4] Learn About CPU Scheduling (Educational Mode)
[5] Exit Program

Your choice (1-5): 1
```

```

Your choice (1-5): 1

>> MANUAL PROCESS ENTRY <<

-----

[i] You will now enter the details of each process (program).
[i] For each process, you need to provide:

    * PID      : A unique ID number for the process (e.g., 1, 2, 3)
    * Arrival  : When the process arrives (time unit, 0 = start)
    * Burst    : How long the process needs the CPU (time units)
    * Priority  : Importance level (1 = highest, larger = lower)

How many processes? (1-20): 1

--- Process 1 of 1 ---
Enter PID (unique number, 1-999): 1
Arrival time (0-1000): 2
Burst time (1-500): 3
Priority (1-100): 1
[OK] Process P1 added successfully!

[OK] All 1 processes entered!

+-----+
| PID | Arrival | Burst | Priority |
+-----+
| 1 | 2 | 3 | 1 |
+-----+

>> ROUND ROBIN TIME QUANTUM <<

-----

[i] Round Robin gives each process a fixed time slice.
[i] A smaller quantum = fairer but more switches.
[i] A larger quantum = fewer switches but less responsive.

Enter time quantum (1-20, recommended 2-4): 5

=====
RUNNING ALL SCHEDULING ALGORITHMS
=====

[i] Running 6 scheduling algorithms on your data...

[1/6] Running FCFS (First Come, First Served)...
[OK] FCFS complete.
[2/6] Running SJF (Shortest Job First)...
[OK] SJF complete.
[3/6] Running SRTF (Shortest Remaining Time First)...
[OK] SRTF complete.
[4/6] Running Round Robin (quantum=5)...
[OK] Round Robin complete.
[5/6] Running Priority Scheduling (with Aging)...
[OK] Priority complete.
[6/6] Running Adaptive AI Scheduler...

>> AI WORKLOAD ANALYSIS <<

-----

[i] Analyzing workload characteristics...

Number of processes : 1
Mean burst time      : 3.00
Burst time Std Dev   : 0.00
Burst time CV        : 0.00
Priority Std Dev      : 0.00
Arrival time spread  : 0.00

[i] Algorithm Scores:

FCFS      [ 60 pts] ***** <-- WINNER
SJF        [ 30 pts] *****
SRTF        [  0 pts]
Round Robin [ 35 pts] *****
Priority     [  0 pts]

[i] Reasoning for selection: FCFS
    * Burst times are very similar (CV=0.00) -> FCFS is fair
    * Priorities are nearly identical -> priority scheduling not needed
    * Few processes (1) -> FCFS is simple and effective
    * Simultaneous arrival -> FCFS is straightforward
[OK] Adaptive AI complete.

Press Enter to continue...

=====
GANTT CHARTS
=====

[i] Each chart shows which process runs at each time unit.
[i] Colored blocks = process running, IDLE = CPU not busy.

Gantt Chart: FCFS
+-----+
| IDLE | 1 | 2 | 3 |
+-----+
0      2      5

Gantt Chart: SJF
+-----+
| IDLE | 1 | 2 | 3 |
+-----+
0      2      5

Gantt Chart: SRTF
+-----+
| IDLE | 1 | 2 | 3 |
+-----+
0      2      5

Gantt Chart: RR (q=5)
+-----+
| IDLE | 1 | 2 | 3 |
+-----+
0      2      5

Gantt Chart: Priority
+-----+
| IDLE | 1 | 2 | 3 |
+-----+
0      2      5

Gantt Chart: AI (FCFS)
+-----+
| IDLE | 1 | 2 | 3 |
+-----+
0      2      5

Press Enter to continue...
```

```

=====
DETAILED PER-PROCESS RESULTS
=====
[1] Showing metrics for each process under each algorithm.

Per-Process Results: FCFS
=====
| PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting | Response |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2 | 3 | 1 | 5 | 3 | 0 | 0 |
|-----|-----|-----|-----|-----|-----|-----|
Avg: 3.00 0.00 0.00

Per-Process Results: SJF
=====
| PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting | Response |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2 | 3 | 1 | 5 | 3 | 0 | 0 |
|-----|-----|-----|-----|-----|-----|-----|
Avg: 3.00 0.00 0.00

Per-Process Results: SRTF
=====
| PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting | Response |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2 | 3 | 1 | 5 | 3 | 0 | 0 |
|-----|-----|-----|-----|-----|-----|-----|
Avg: 3.00 0.00 0.00

Per-Process Results: RR (q=5)
=====
| PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting | Response |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2 | 3 | 1 | 5 | 3 | 0 | 0 |
|-----|-----|-----|-----|-----|-----|-----|
Avg: 3.00 0.00 0.00

Per-Process Results: Priority
=====
| PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting | Response |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2 | 3 | 1 | 5 | 3 | 0 | 0 |
|-----|-----|-----|-----|-----|-----|-----|
Avg: 3.00 0.00 0.00

Per-Process Results: AI (FCFS)
=====
| PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting | Response |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2 | 3 | 1 | 5 | 3 | 0 | 0 |
|-----|-----|-----|-----|-----|-----|-----|
Avg: 3.00 0.00 0.00

Press Enter to continue...

```

```

=====
COMPARATIVE METRICS TABLE
=====
Metric      FCFS      SJF      SRTF      RR (q=5)  Priority  AI (FCFS)
-----
Avg Wait Time  0.00*    0.00*    0.00*    0.00*    0.00*    0.00*
Avg Turnaround 3.00*    3.00*    3.00*    3.00*    3.00*    3.00*
Avg Response   0.00*    0.00*    0.00*    0.00*    0.00*    0.00*
CPU Utilization 60.0%    60.0%    60.0%    60.0%    60.0%    60.0%
Throughput     0.2000*  0.2000*  0.2000*  0.2000*  0.2000*  0.2000*
Context Switches 1*       1*       1*       1*       1*       1*
Total Time     5        5        5        5        5        5

* = Best value in that metric

=====
OVERALL WINNER: FCFS
=====

This algorithm scored the best overall across all metrics.
Waiting time and turnaround time are weighted highest in the scoring.

Press Enter to continue...

```

```

=====
WHAT WOULD YOU LIKE TO DO NEXT?
=====
[1] Run again with different data
[2] Learn about CPU scheduling concepts
[3] Exit program

Your choice (1-3): 3

[OK] Thank you for using the CPU Scheduler Simulator!
Goodbye!

PS C:\Users\Upa>

```

SIMULATION SCENARIOS:

Scenario 1 (Mixed Workload): 8 processes with varied bursts and priorities. The AI correctly identified moderate burst variance and selected Round Robin to ensure fairness. SRTF won the comparative metrics by minimizing turnaround time. -

Scenario 2 (Simultaneous Arrival): 5 processes all arriving at time 0.

Scenario 3 (Starvation Risk): 6 processes with extreme priority gaps. The Priority algorithm successfully executed aging, gradually elevating the priority of starved processes and completing them before higher-priority late arrivals.

Comparative Metrics Output: In the executed tests, the program successfully rendered a side-by-side table of all 6 algorithms, displaying Average Wait Time, Average Turnaround, Average Response, CPU Utilization, Throughput, and Context Switches. The lowest/best values were correctly highlighted with asterisks, and the overall optimal algorithm was identified algorithmically.

DISCUSSION OF RESULTS

The results demonstrate that the proposed C++ architecture is practical for educational OS simulation and aligns perfectly with theoretical scheduling principles. The decoupled trace-array model is transparent and predictable, which is valuable for verifying manual calculations where the user needs to know exactly which process ran at which millisecond.

The Adaptive AI is the strongest reliability feature in the implementation. Instead of forcing a user to guess the best approach, the system actively prevents convoy effects (by avoiding FCFS on high-variance workloads) and starvation (by avoiding strict Priority on dense workloads without warning). The testing also shows that preemptive algorithms generate significantly more context switches, accurately reflecting real-world CPU overhead.

From a systems perspective, the solution effectively demonstrates the use of operating-system concepts through a streamlined application: process control blocks (PCBs), ready queues, context switching, preemption, and statistical resource management.

CONCLUDING REMARKS

The AI-Based Adaptive CPU Scheduler successfully converts tedious mathematical process scheduling into a highly visual, automated simulation workflow. Its modular structure separates user input, algorithmic execution, and metric computation, while the educational mode keeps the user in control of understanding the underlying concepts. The practical tests confirm mathematically accurate Gantt generation, correct collision-aware preemption, and successful algorithmic comparisons. Overall, the project demonstrates how core C++ data structures can be combined into a lightweight, auditable, and highly educational simulation tool for operating system theory.